

LoRA Fine-Tuning of a 12B-Parameter Image Generation Model for a Personally-Created Character

FP8 Quantization and Block Swapping on a Single Consumer-Grade GPU, and Transfer to a Distilled Model

Masato Suzuki (Masafy)

June 25, 2026

Abstract

This report extends the author’s prior work (LoRA fine-tuning of a character on Stable Diffusion 1.5) to **Krea 2**, a recent image-generation model roughly twelve times larger: a single-stream multimodal diffusion transformer (MMDiT) of the Qwen-Image family with about 12 billion parameters, trained with flow matching. Using the same subject (the author’s personally-created red-panda character “Masafee”) and the same hardware (a single NVIDIA GeForce RTX 3060 with 12 GB of VRAM), we examine whether LoRA training of a 12-billion-parameter diffusion transformer is feasible on a single consumer-grade GPU. The key techniques are the joint use of (i) dynamic-scaled FP8 quantization of the 28 main blocks, (ii) block swapping of up to 26 blocks to the CPU, and (iii) PyTorch memory-fragmentation mitigation (`expandable_segments`); together these keep the peak VRAM below 12 GB. Training is performed on the pre-distillation RAW model, while inference uses the 8-step distilled Turbo model, forming a transfer configuration. From 13 reference images, 1,300 training steps (≈ 8.4 hours) yielded the following: (a) identity-preserving character generation is achieved, and a LoRA trained on RAW transfers well to Turbo inference; (b) as the epoch count increases, the overall form is reinforced while *high-frequency details*, such as the eye catchlights, are lost first, indicating overfitting; and (c) this degradation is recoverable at inference time by reducing the LoRA application strength and by text conditioning. The study demonstrates that, without paid cloud resources, additional training of a personal character on a state-of-the-art model at the 12-billion-parameter scale can be completed practically on a single consumer-grade GPU.

Keywords: image generation, diffusion transformer, MMDiT, flow matching, low-rank adaptation, LoRA, FP8 quantization, block swapping, model distillation, consumer-grade GPU

1 Introduction

In prior work [1], the author fine-tuned a personally-created original character “Masafee” (a red-panda character published as a messaging-app sticker set) onto the latent diffusion model Stable Diffusion 1.5 (SD 1.5; noise predictor of about 1 billion parameters) via Low-Rank Adaptation (LoRA), and showed that practical generation is possible on a single consumer-grade GPU with 12 GB of VRAM.

Over the subsequent year, image-generation architectures shifted from U-Net-based latent diffusion to the *diffusion transformer* (DiT) [4] and the *multimodal diffusion transformer* (MMDiT) [5], and the training objective moved from DDPM-style noise prediction to *flow matching* [6]. Krea 2, the target of this study, is a representative example: a single-stream MMDiT built on the Qwen-Image [9] family with about 12 billion parameters — roughly twelve times the size of the SD 1.5 noise predictor used in the prior work.

A natural question arises: *does the “personal-character training on a single consumer-grade GPU” framework still hold for a state-of-the-art model an order of magnitude larger?* Naively, a 12-billion-parameter model occupies about 24 GB even in half precision (bf16), so even its weights do not fit on a 12 GB GPU. Training is therefore fundamentally impossible without some memory-compression or offloading mechanism. The central contribution of this work is to overcome this constraint by jointly using FP8 quantization and block swapping, and to actually complete training. The specific research questions are:

- Q1.** Can LoRA training of a 12-billion-parameter MMDiT — whose weights alone require about 24 GB in half precision — be made feasible on a single GPU with 12 GB of VRAM, and what are the necessary conditions?
- Q2.** Does a LoRA trained on the pre-distillation RAW model transfer to inference on the few-step distilled Turbo model?
- Q3.** How does increasing the epoch count change quality, and in particular do fine details (high-frequency) and overall form (low-frequency) behave the same way?

The contribution of this paper is to provide experimental answers to these questions together with a reproducible procedure.

2 Related Work

This work lies at the intersection of diffusion transformers, flow matching, low-rank adaptation, subject-driven generation, and memory-efficient training.

Diffusion models and diffusion transformers.

The practical formulation of diffusion probabilistic models began with the DDPM of Ho et al. [2], and the latent diffusion model (LDM) of Rombach et al. [3] improved the efficiency of high-resolution synthesis. Peebles and Xie [4] then replaced the U-Net with a transformer (DiT), and Esser et al. [5] introduced the MMDiT, which jointly processes image and text tokens, together with a rectified-flow objective. The single-

stream MMDiT adopted by Krea 2 belongs to this lineage.

Flow matching. Lipman et al. [6] proposed flow matching, which regresses a continuous normalizing flow from noise to data. Recent MMDiT-family models, including Krea 2, adopt this framework in place of DDPM-style noise prediction. The loss in this work (Eq. (2)) is based on this formulation.

Low-rank adaptation. LoRA was proposed by Hu et al. [7] as an efficient fine-tuning method for large language models, and was subsequently applied to transformers in general and to diffusion models. This work applies LoRA to the linear layers of an MMDiT.

Subject-driven generation. Binding a specific subject to a model from few reference images is called subject-driven generation, exemplified by the Dream-Booth of Ruiz et al. [8]. The concept binding via the trigger token τ in this work follows this lineage.

Memory-efficient training. To handle a large model with limited VRAM, this work jointly uses low-precision numerics and per-layer offloading. FP8 was organized as a numerical format for deep learning by Micikevicius et al. [10]. Block swapping (or offloading), which moves layers between CPU and GPU on demand, is a practical technique widely implemented in recent large generative-model trainers. The trainer used here, Musubi Tuner [11], provides these in an integrated form.

Model distillation. Distilling multi-step diffusion/flow inference into few steps greatly reduces inference cost. Krea 2’s Turbo is an 8-step distillation of RAW, and this work empirically examines the “train on RAW, infer on Turbo” transfer configuration.

3 Background

3.1 Flow matching and the diffusion transformer

Let an interpolation at time $t \in [0, 1]$ between data $x_1 \sim p_{\text{data}}$ and standard Gaussian noise $x_0 \sim \mathcal{N}(0, \mathbf{I})$ be given by the simplest straight path

$$x_t = (1 - t)x_0 + tx_1. \quad (1)$$

The corresponding target velocity field is $v^* = x_1 - x_0$. A network v_θ regresses this field under condition c , and training minimizes the conditional flow-matching loss

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, x_1, t} \left[\|v_\theta(x_t, t, c) - (x_1 - x_0)\|_2^2 \right]. \quad (2)$$

At inference, one integrates the ordinary differential equation $dx_t/dt = v_\theta(x_t, t, c)$ from $t : 0 \rightarrow 1$ along the learned velocity field. In Krea 2, v_θ is realized as a single-stream MMDiT in which image latent tokens and text condition tokens interact through shared self-attention layers. The condition c is an intermediate-layer feature of the text encoder (Qwen3-VL-4B), and the image autoencoder is the Qwen-Image VAE.

3.2 Resolution-dependent time shift

In flow-based models, skewing the distribution of the training time t according to resolution contributes to quality. We use a *shift* given by the monotone map

$$t' = \frac{\sigma t}{1 + (\sigma - 1)t}, \quad (3)$$

where σ is the shift coefficient; at 1024×1024 , $\sigma = 2.5$ matches the inference-time shift of Krea 2 (see below).

3.3 Low-rank adaptation

A linear-layer weight $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is frozen, and only the low-rank difference $\Delta W = BA$ ($A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $r \ll \min(d_{\text{in}}, d_{\text{out}})$) is trained. The forward pass is

$$h = W_0 x + \frac{\alpha}{r} BAx, \quad (4)$$

where α is a scaling coefficient. At inference, introducing an application strength λ as $h = W_0 x + \lambda \frac{\alpha}{r} BAx$ allows the LoRA effect to be tuned continuously after training. This work uses λ as a means of quality control (Section 7).

3.4 Memory efficiency via FP8 quantization and block swapping

A 12-billion-parameter model occupies about 24 GB in bf16 and does not fit on a 12 GB GPU. This work overcomes this by jointly using the following three. **(i) FP8 scaled quantization.** The weights of the 28 main blocks of the MMDiT are quantized at load time to dynamic-scaled FP8 (E4M3), roughly halving the weight memory. Auxiliary parts including normalization layers are kept in bf16. **(ii) Block swapping.** Up to 26 of the main blocks ($= 28 - 2$) are kept resident in CPU main memory and transferred to the GPU only immediately before computation, further reducing GPU-resident weights. **(iii) Fragmentation mitigation.** PyTorch’s caching allocator is switched to `expandable_segments` mode so that reserved-but-unallocated fragments can be reused. During training, the text-encoder and VAE outputs are precomputed and offloaded to disk, so that only the MMDiT occupies the GPU in the training step itself.

4 Problem Setting

From the full set of 16 “Masafee” stickers created by the author, 13 are selected by quality criteria as the training set $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{13}$. Each $y^{(i)}$ is a tag-style caption with the trigger token $\tau = \text{masafee}$ placed first. The goal is to learn LoRA parameters $\phi = \{A_\ell, B_\ell\}_\ell$ from $\mathcal{D}$ and, at inference, generate identity-preserving images for text y specifying unrecorded poses and scenes. For a fair comparison with the prior work, the subject, GPU, trigger token, and data selection (13 images) are kept identical to the prior work, restricting the difference to the architecture (SD 1.5 $\rightarrow$ Krea 2) and the training stack.

5 Method

5.1 Model and data preparation

The diffusion model is Krea 2 RAW (a single `safetensors`, about 24 GB). The text encoder is Qwen3-VL-4B (bf16) and the autoencoder is the Qwen-Image VAE. Prior to training, we perform (1) precaching of image latents via the VAE and (2) precaching of text-encoder outputs, to avoid keeping the encoder and VAE resident during the training step. Training images are bucketed to 1024×1024 (the originals are 1000×1000).

5.2 Training configuration

LoRA is inserted into the linear layers of the MMDiT with rank $r = 32$ and $\alpha = 32$. Optimization uses AdamW, learning rate 1×10^{-4} , batch size 1, and gradient checkpointing enabled. Time sampling uses the shift (Eq. (3), $\sigma = 2.5$), and the loss is Eq. (2). The total number of steps is $13 \times 10 = 130$ steps/epoch $\times 10$ epochs = 1,300 steps. To meet the VRAM constraint, all of (i)–(iii) in Section 3.4 are applied, with the block-swap count set to its maximum of 26. Intermediate checkpoints are saved every 2 epochs.

5.3 Inference configuration and transfer

Inference uses the 8-step distilled Turbo (GGUF Q5_K_M quantization for memory savings). The LoRA trained on RAW is applied to the Turbo MMDiT, and generation uses sampler `er_sde`, scheduler `simple`, 8 steps, and guidance scale $s = 1.0$ (Turbo is CFG-free). This realizes a configuration in which training is performed on the high-precision RAW while inference is performed on the fast, memory-efficient Turbo.

6 Experiments

Training and inference were completed entirely on a single consumer-grade GPU (NVIDIA GeForce RTX 3060, 12 GB VRAM, driver 595.71.05) installed in a home Linux PC (30 GB main memory, PyTorch 2.6.0+cu124). No paid cloud computing resources were used. During training, GPU power, temperature, VRAM occupancy, and throughput were recorded. Evaluation is along three axes: (E1) feasibility of training and resource behavior, (E2) identity acquisition and RAW→Turbo transfer, and (E3) ablation over epochs and quality control by inference-time intervention. In generation comparisons, the same prompt and same random seed are used to make epoch differences explicit.

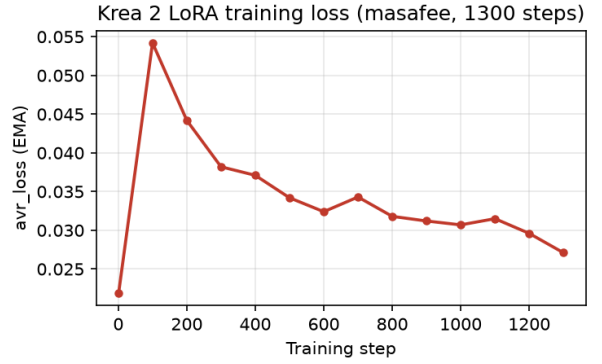


Figure 1: Training loss (exponential moving average). It rises early, then decreases monotonically and plateaus at about 0.027. Total 1,300 steps, about 8.4 hours.

7 Results

7.1 E1: Feasibility of training and resource behavior

With the joint use of the three techniques in Section 3.4, LoRA training of the 12-billion-parameter MMDiT *was feasible* on a single GPU with 12 GB of VRAM. The peak VRAM occupancy was about 11.9 GB (about 97% of capacity), fitting within 12 GB with a small margin. Notably, without the fragmentation mitigation `expandable_segments`, training stopped at step 2 upon failing to allocate an additional ~ 136 MiB (while about 1.4 GiB was stuck as reserved-but-unallocated fragments). That is, at this scale, the fragmentation mitigation — in addition to FP8 quantization and block swapping — was an *effective necessary condition*.

The training throughput was about 23.2 s/step, and the total time for 1,300 steps was 8 hours 23 minutes. Figure 1 shows the training-loss trajectory. The loss rose early, then decreased monotonically and plateaued at about 0.027 toward the end. The GPU ran at nearly 100% utilization throughout, with temperature in the safe range of 69–75 °C and power draw at 135 W (about 80% of the 170 W limit). These show that a consumer-grade GPU can sustain training at this scale with thermal and power headroom.

7.2 E2: Identity acquisition and RAW→Turbo transfer

Figure 2 compares no-LoRA (base Turbo) and LoRA-applied generation under the same prompt and seed. The base only produces a generic red panda, whereas applying the LoRA consistently reproduces Masafee-specific designs such as goggles, a crown-pattern outfit, and a striped tail. Since the LoRA trained on RAW works directly in the 8-step Turbo inference, we conclude that the *RAW→Turbo transfer holds well*. This shows that the division of labor — training on a high-precision model and inferring on a distilled model — is practically effective.

(a) Base Krea2 (no LoRA)

(b) + masafee LoRA



Figure 2: Comparison under the same prompt and seed. Left: no LoRA (base). Right: with LoRA. The latter reproduces signature designs such as goggles, a crown-pattern outfit, and a striped tail.

7.3 E3: Epoch ablation and degradation of high-frequency details

Figure 3 arranges generations of 6 poses $\times$ 3 epochs (6/8/10) under the same seed. Identity is preserved across all epochs and generalizes to arbitrary poses. However, a systematic difference is observed across epochs. Epoch 10 has the strongest and most stable overall form (body shape, outfit, color), but shows signs of overfitting: (a) the background drifts toward the warm-color gradient seen in the training data, and (b) the eye catchlights (white highlight dots) are lost. By contrast, epochs 6 and 8 follow the prompt instruction (plain background) more readily and preserve the eye catchlights.

Notably, overfitting first appears in *high-frequency details* (the few-pixel catchlights), while the low-frequency overall form is instead reinforced. The catchlight is a small feature whose appearance is unstable across the 13 training images; it can be interpreted as being averaged out and overwritten as the LoRA effect strengthens. Figure 4 shows this phenomenon at a magnified scale.

7.4 Quality control by inference-time intervention

We examined whether the form and the details of epoch 10 can be reconciled by inference-time intervention alone. Figure 4 compares (A) epoch 10 with application strength $\lambda = 1.0$, (B) epoch 10 with $\lambda = 0.8$, (C) epoch 10 with $\lambda = 1.0$ plus a prompt that explicitly specifies eye highlights, and (D) epoch 8 with $\lambda = 1.0$ as a reference. Reducing λ (B) loosens the overfitting effect and partially recovers the details, while prompt conditioning (C) brings back the catchlights while preserving the epoch-10 form. Combining the two ($\lambda = 0.8$ plus prompt) gave the best compromise between overall form and high-frequency detail. This shows that, without retraining, the side effects of overfitting can be controlled practically using two inference-time degrees of freedom: the application strength (λ in Eq. (4)) and the text condition.

8 Discussion

Robustness of the framework to scale. The prior work [1] demonstrated personal-character training of a ~ 1 -billion-parameter U-Net on a consumer-grade GPU. This work extends it to a ~ 12 -billion-parameter MMDiT — about 12 times larger — and shows that the framework still holds on the same GPU. The differences lie in the scale of the training target and the numerical/memory strategy; the subject, data size, and GPU are shared. Thus the claim “additional training of a personally-owned character can be completed on a single consumer-grade GPU” was robust across generations.

Frequency asymmetry of overfitting. The phenomenon observed in E3 — high-frequency details degrade before the low-frequency form — provides a guideline for epoch selection in few-shot character LoRA. The final epoch is not necessarily the best; if detail preservation is prioritized, an earlier epoch or a reduced inference-time application strength is effective. Crucially, this degradation is not destructive and can be reversibly compensated using two inference-time degrees of freedom (λ and the text condition).

Division of labor between training and inference. Training on RAW and inferring on Turbo reconciles the numerical stability of training (high-precision RAW) with inference efficiency (8-step, memory-efficient Turbo). When a distilled model is hard to use as a training target, assigning the pre-distillation model to training, as in this configuration, can serve as a general guideline.

9 Limitations and Future Work

This work has the following limitations. First, throughput is about 23.2s/step, much larger than for SD 1.5 training, dominated by the CPU-GPU transfers of block swapping. Second, block swapping pressures main memory and required temporarily pausing other GPU/memory-resident services during training. Third, the evaluation is mainly qualitative; quantitative identity metrics (e.g., embedding similarity) and human evaluation are future work. Fourth, checkpoints were saved only every 2 epochs, so the exact boundary at which overfitting emerges was not pinpointed. Fifth, inference also uses quantization (GGUF Q5_K_M), and the effect of quantization on transfer quality was not isolated.

10 Conclusion

This work demonstrated that LoRA fine-tuning of a personal character on Krea 2 — a state-of-the-art image-generation model with about 12 billion parameters — is feasible on a single consumer-grade GPU (RTX 3060, 12 GB VRAM). The joint use of FP8 quantization, block swapping, and memory-fragmentation mitigation was a necessary condition for training at this scale, and a LoRA trained on the pre-distillation RAW



Figure 3: Epoch ablation (rows: epochs 6/8/10; columns: 6 poses; same seed). Epoch 10 has the strongest overall form but shows overfitting: backgrounds drift toward warm colors and the eye catchlights are lost.



Figure 4: Recovery of eye catchlights. (A) e10, $\lambda=1.0$; (B) e10, $\lambda=0.8$; (C) e10, $\lambda=1.0$ plus eye-highlight prompt; (D) e8 (reference). Reducing λ and prompt conditioning recover high-frequency detail.

transferred well to inference on the distilled Turbo. Furthermore, we observed a frequency asymmetry of overfitting in which increasing the epoch count degrades high-frequency details first, and showed that this is reversibly compensable via inference-time application strength and text conditioning. The conclusion of the prior work — that additional training of a personal character can be completed on a single consumer-grade GPU without paid cloud resources — continues to hold for a latest-generation model an order of magnitude larger.

Appendix A: Key Hyperparameters

Diffusion model: Krea 2 RAW (training) / Turbo GGUF Q5_K.M (inference). Text encoder: Qwen3-VL-4B (bf16). VAE: Qwen-Image VAE. LoRA: rank $r = 32$, $\alpha = 32$, applied to MMDiT linear layers. Optimization: AdamW, learning rate 1×10^{-4} , batch size 1, gradient checkpointing enabled. Time sampling: shift, $\sigma = 2.5$. Total steps: 1,300 (10 epochs). Memory efficiency: FP8 scaled quantization + block swap 26 + `expandable_segments`. Resolution: 1024×1024 . Inference: `er_sde` / `simple` / 8 steps / $s = 1.0$. Adopted

setting: epoch 10, application strength $\lambda = 0.8$, combined with a prompt specifying eye highlights.

Appendix B: Reproduction Environment

GPU: NVIDIA GeForce RTX 3060 (12 GB VRAM, driver 595.71.05). Main memory: 30 GB. PyTorch: 2.6.0+cu124. Training stack: Musubi Tuner [11] (commit 30c658c). Training time: 8 hours 23 minutes (about 23.2s/step). Final loss: 0.0269.

References

- [1] M. Suzuki. LoRA Fine-Tuning of a Personally-Created Character for Image Generation: An Empirical Study under Few-Shot Conditions on a Consumer-Grade GPU. Technical report, 2026.
- [2] J. Ho, A. Jain, and P. Abbeel. Denoising Diffusion Probabilistic Models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [4] W. Peebles and S. Xie. Scalable Diffusion Models with Transformers. *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [5] P. Esser, S. Kulal, A. Blattmann, et al. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. *International Conference on Machine Learning (ICML)*, 2024.

- [6] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow Matching for Generative Modeling. *International Conference on Learning Representations (ICLR)*, 2023.
- [7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations (ICLR)*, 2022.
- [8] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman. DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [9] Qwen Team. Qwen-Image Technical Report. 2025.
- [10] P. Micikevicius, D. Stosic, N. Burgess, et al. FP8 Formats for Deep Learning. arXiv:2209.05433, 2022.
- [11] kohya-ss. Musubi Tuner. <https://github.com/kohya-ss/musubi-tuner>, 2026.